

Scientific Mesh Intelligence: Ecosystem Architecture

1. mesh-core

Purpose: Core logic for mesh processing, geometric feature engineering, and graph construction.

Why it exists: We cannot train models without transforming continuous meshes into discrete graph structures with computed geometric features. This repo centralizes that math. **Why it matters:** A highly optimized, unified library prevents duplicated effort across teams and ensures consistency in data representation—it is the backbone of all our ML pipelines. **Public API:** `load_mesh()`, `compute_normals()`, `extract_geometric_features()`, `to_pyg_graph()`, `coarsen_mesh()`.

Folder Structure:

```
mesh-core/
├─ src/
│   ├── geometry/ (normals, curvature, geodesics)
│   ├── graph/    (graph construction, coarsening)
│   └─ io/        (read/write obj, stl, vtk)
├─ tests/
└─ csrc/          (C++ optimizations)
```

Dependencies: `numpy`, `scipy`, `trimesh`, `networkx`, `pybind11`. **Integration Points:** Imported by `mesh-ml`, `mesh-data`, and `mesh-vis`. **Release Strategy:** Semantic versioning (e.g., v1.x.x), auto-published to PyPI on GitHub releases. **Implementation Strategy:** Python frontend with C++ optimizations for bottleneck algorithms (like KD-tree lookups or large-scale graph coarsening).

Repository Ownership: Core Engineering Team. **Migration Plan:** Greenfield implementation. Build from scratch ensuring strict typing and performance, avoiding technical debt from day one. **Expected Impact:** Standardized mesh representation across the company and 10x faster data processing pipelines. **Risks:** Cross-platform C++ build complications (Windows/Linux/Mac). **Priority:** P0 (Foundational) **Estimated Engineering Effort:** 2-3 months for a stable v1.0. **Future Roadmap:** GPU-accelerated mesh processing (CUDA) and out-of-core processing for massive industrial meshes (>100M elements).

2. mesh-ml

Purpose: Houses the neural network architectures, including GNNs, graph transformers, foundation models, and physics-informed learning logic. **Why it exists:** Isolates complex ML research and model definitions from data processing and deployment, providing a clean framework for model development. **Why it matters:** This is the intelligence engine of the organization, directly responsible for predicting structural wear patterns, stress, and scientific phenomena. **Public API:** `MeshGNN`, `GraphTransformer`, `WearPredictionModel`, `PhysicsInformedLoss`, `train_step()`. **Folder Structure:**

```
mesh-ml/
├─ src/
│   ├── foundation_models/
│   ├── transformers/
│   └─ gnns/
```

```
|   └─ losses/ (Physics-informed)
└─ tests/
```

Dependencies: `torch`, `torch-geometric`, `mesh-core`. **Integration Points:** Consumes `mesh-core` for data. Trained models are exported to `mesh-hub` and served by `mesh-serve`. **Release Strategy:** Tied to major model breakthroughs or foundation model checkpoint releases. **Implementation Strategy:** PyTorch-centric. Abstract base classes for layers to allow easy mixing of Message Passing and Transformer attention mechanisms. **Repository Ownership:** AI Research & Applied ML Team. **Migration Plan:** Greenfield. Separate model architecture definitions cleanly from training loops and infrastructure. **Expected Impact:** Rapid iteration on state-of-the-art graph/mesh AI models. **Risks:** Training instability with complex physics-informed losses; memory OOM issues when scaling Graph Transformers. **Priority:** P0 (Core Value) **Estimated Engineering Effort:** 4-6 months to establish the first foundation models. **Future Roadmap:** Linear-complexity attention mechanisms for meshes, and multi-modal integration (e.g., mesh + text prompts).

3. mesh-data

Purpose: Tools for dataset generation, procedural mesh synthesis, data augmentation, and standardized benchmark suites. **Why it exists:** Machine learning requires vast, clean datasets. This repo standardizes how we generate data and evaluate our models. **Why it matters:** High-quality scientific data is our defensive moat. Objective benchmarks are strictly required to measure genuine model improvements. **Public API:** `generate_wear_dataset()`, `augment_mesh()`, `run_benchmark(model)`, `evaluate_metrics()`. **Folder Structure:**

```
mesh-data/
├─ src/
│   ├── generators/
│   ├── augmentations/
│   └─ benchmarks/
├─ datasets/
└─ scripts/
```

Dependencies: `mesh-core`, `huggingface_hub`, `pytest`, `mesh-sim`. **Integration Points:** Feeds training data into `mesh-ml`. Uses `mesh-sim` to generate ground truth physical labels. **Release Strategy:** Nightly data generation cron jobs; monthly stable benchmark suite releases. **Implementation Strategy:** Procedural generation scripts utilizing multiprocessing; deep integration with HuggingFace Datasets for large-scale blob storage. **Repository Ownership:** Data Engineering Team. **Migration Plan:** Greenfield. Centralize all disparate data generation scripts into a cohesive pipeline. **Expected Impact:** Unlocks infinite training data generation; establishes the industry-standard benchmark for Scientific Mesh AI. **Risks:** Runaway cloud compute and storage costs for continuous dataset generation. **Priority:** P1 **Estimated Engineering Effort:** 3 months. **Future Roadmap:** Generative AI for synthetic mesh generation (e.g., 3D diffusion models for mesh topology).

4. mesh-sim

Purpose: Simulation interfaces connecting meshes to physical solvers (FEM, CFD) for ground-truth generation and physics-informed training. **Why it exists:** Bridges the gap between AI and traditional physical simulations. **Why it matters:** Provides the physical accuracy, constraints, and ground-truth

data necessary to make our AI "Scientific". **Public API:** `run_fem_solver()` , `extract_boundary_conditions()` , `differentiable_physics_step()` . **Folder Structure:**

```
mesh-sim/
├─ src/
│   ├── interfaces/
│   │   ├── ansys/
│   │   └── openfoam/
│   └── differentiable/
└─ tests/
```

Dependencies: `mesh-core` , `fenics` , external solvers (Ansys, OpenFOAM). **Integration Points:**

Used by `mesh-data` for label generation and `mesh-ml` for calculating physics-informed losses.

Release Strategy: Quarterly releases aligned with major external solver tool updates.

Implementation Strategy: Python wrappers around solver APIs; custom PyTorch autograd functions for differentiable physics bridging. **Repository Ownership:** Physics & Simulation Team. **Migration Plan:** Greenfield. Build abstract interfaces to avoid vendor lock-in to a single commercial solver.

Expected Impact: Allows seamless comparison and integration of AI predictions with ground-truth physics. **Risks:** Restrictive licensing issues with commercial solvers; API breaking changes from upstream simulation tools. **Priority:** P1 **Estimated Engineering Effort:** 4-5 months. **Future Roadmap:** Fully differentiable, GPU-native physics solvers written entirely in PyTorch/JAX to bypass legacy software.

5. `mesh-vis`

Purpose: Visualization tools, including an interactive viewer (WebGL/Desktop) and Python plotting for meshes and tensor fields. **Why it exists:** 3D scientific data and AI predictions (like wear and stress) are impossible to interpret without specialized visual tooling. **Why it matters:** Critical for debugging models, presenting to stakeholders, and building user trust. **Public API:** `plot_mesh()` , `visualize_stress()` , `launch_viewer()` . **Folder Structure:**

```
mesh-vis/
├─ src/
│   ├── python_plot/
│   └── viewer_app/ (React/Three.js)
└─ tests/
```

Dependencies: `mesh-core` , `pyvista` , `vtk` , `three.js` (for web). **Integration Points:** Visualizes outputs from `mesh-ml` , `mesh-sim` , and `mesh-core` . **Release Strategy:** Weekly updates for the web viewer; semantic versioning for the Python plotting package. **Implementation Strategy:** PyVista for quick Python notebook plotting; React + Three.js/WebGL for a robust, shareable web viewer.

Repository Ownership: Frontend & Visualization Team. **Migration Plan:** Greenfield. Discard ad-hoc matplotlib scripts in favor of a unified 3D rendering pipeline. **Expected Impact:** Beautiful, intuitive understanding of complex AI outputs; vital for sales, research, and debugging. **Risks:** Browser memory limits and rendering bottlenecks when handling meshes with millions of polygons. **Priority:** P2 **Estimated Engineering Effort:** 3 months. **Future Roadmap:** Collaborative WebXR viewer for remote, multi-user mesh inspection.

6. mesh-serve

Purpose: Production deployment tools, including REST APIs, SDKs, CLI, and Docker containerization.

Why it exists: Models must be accessible to end-users and other systems in a scalable, reliable way.

Why it matters: This is the primary interface for external customers and production pipelines to consume our AI, driving revenue and adoption. **Public API:** REST /predict , Python SDK

MeshClient , CLI mesh-serve run . **Folder Structure:**

```
mesh-serve/  
├─ api/ (FastAPI)  
├─ sdk/  
├─ cli/  
├─ docker/  
└─ tests/
```

Dependencies: fastapi , mesh-core , mesh-ml , docker , onnxruntime . **Integration Points:**

Wraps mesh-ml models and serves them over the network. **Release Strategy:** CI/CD pipeline deploying Docker images on merge; strict backward compatibility for SDKs and APIs. **Implementation Strategy:** FastAPI for high-throughput serving; ONNX/TensorRT for optimized, low-latency inference.

Repository Ownership: MLOps / Platform Team. **Migration Plan:** Greenfield. **Expected Impact:**

Highly available, scalable model inference for enterprise clients. **Risks:** Large 3D mesh payloads

causing network bottlenecks or OOM errors on inference servers. **Priority:** P1 **Estimated**

Engineering Effort: 3 months. **Future Roadmap:** Edge deployment capabilities (e.g., C++ SDKs) for running models offline on local machines.

7. mesh-hub

Purpose: Centralized platform hosting the Model Zoo, Leaderboards, and community model sharing.

Why it exists: To foster community engagement and provide a centralized registry for all trained weights and evaluation metrics. **Why it matters:** A model zoo accelerates adoption. Public

leaderboards drive competition and establish us as the authoritative platform for Mesh AI. **Public API:**

download_model() , submit_leaderboard() , Web UI. **Folder Structure:**

```
mesh-hub/  
├─ web/ (Next.js App)  
├─ registry/ (Model tracking logic)  
└─ scripts/
```

Dependencies: mesh-serve , mesh-ml , aws-s3 . **Integration Points:** Front-end for the

benchmarks produced in mesh-data . **Release Strategy:** Continuous deployment for the web

interface. **Implementation Strategy:** Next.js frontend, backed by a lightweight database tracking

model metrics, referencing weights in S3. **Repository Ownership:** Developer Relations & Fullstack

Team. **Migration Plan:** Greenfield. **Expected Impact:** Creates an ecosystem and community around

our tools, driving a viral adoption loop. **Risks:** Unbounded storage and egress costs for hosting

hundreds of large model checkpoints. **Priority:** P2 **Estimated Engineering Effort:** 2 months. **Future**

Roadmap: HuggingFace-style community uploads and automated third-party model evaluation.

8. mesh-docs

Purpose: Centralized documentation, examples, tutorials, and API references. **Why it exists:** To educate users, reduce onboarding friction, and provide clear usage guides. **Why it matters:** Even the best technology fails if developers cannot figure out how to use it. **Public API:** N/A (Web portal).

Folder Structure:

```
mesh-docs/  
├─ docs/ (Markdown/MDX)  
├─ tutorials/ (Jupyter Notebooks)  
└─ examples/ (Standalone scripts)
```

Dependencies: sphinx , mkdocs , jupyter . **Integration Points:** Automatically pulls docstrings and type hints from all other mesh-* repositories. **Release Strategy:** Automated deployment to docs.smi.ai on every commit to main . **Implementation Strategy:** MkDocs Material for clean, searchable documentation; Sphinx for API reference generation. **Repository Ownership:** Tech Writing & DevRel. **Migration Plan:** Greenfield. Establish a strict "docs-as-code" culture from day one. **Expected Impact:** Drastically reduced support burden; happier users and faster onboarding. **Risks:** Documentation drifting out of sync with code (mitigated by CI doc-tests). **Priority:** P1 **Estimated Engineering Effort:** 1-2 months. **Future Roadmap:** Interactive browser-based tutorials (e.g., powered by Pyodide/WebAssembly).

9. mesh-plugins

Purpose: Integrations and plugins for third-party CAD and simulation software (e.g., Blender, Ansys, Maya). **Why it exists:** Engineers want AI insights inside the tools they already use every day, without context switching. **Why it matters:** Dramatically lowers the barrier to entry for non-ML mechanical engineers to utilize our models. **Public API:** Host-specific plugin APIs. **Folder Structure:**

```
mesh-plugins/  
├─ blender/  
├─ ansys/  
└─ maya/
```

Dependencies: mesh-serve (SDK), host software APIs. **Integration Points:** Acts as a thin client, calling mesh-serve to get predictions and rendering them natively in the host application. **Release Strategy:** Independent release cycles per plugin, matching host software updates. **Implementation Strategy:** Python or C++ plugins depending on the host software. **Repository Ownership:** Integration Engineering Team. **Migration Plan:** Greenfield. Start with Blender (open source/accessible) before moving to commercial CAD. **Expected Impact:** Deep penetration into traditional engineering workflows. **Risks:** Maintaining compatibility across multiple OSES and varying versions of proprietary host software. **Priority:** P3 **Estimated Engineering Effort:** 6+ months (ongoing per plugin). **Future Roadmap:** Real-time "AI Co-pilot" for CAD design, predicting wear patterns dynamically as the user modifies the mesh.

10. mesh-research

Purpose: Sandbox for open-source papers, bleeding-edge experiments, and unpolished research code. **Why it exists:** Provides a space for scientists to move fast and break things without compromising the stability of our production repositories. **Why it matters:** Attracts top AI talent,

encourages publications, and drives the long-term innovation of the company. **Public API:** N/A. **Folder Structure:**

```
mesh-research/  
├─ papers/  
│   ├─ 2024_wear_prediction/  
│   └─ 2025_graph_transformers/  
└─ experiments/
```

Dependencies: Unrestricted. **Integration Points:** Successful concepts are formalized and eventually migrated into mesh-ml or mesh-core . **Release Strategy:** Ad-hoc, tied to conference submissions (e.g., NeurIPS, ICLR). **Implementation Strategy:** Monorepo of isolated projects and experiments. **Repository Ownership:** AI Research Team. **Migration Plan:** Greenfield. **Expected Impact:** Breakthroughs in state-of-the-art; high visibility in the academic community. **Risks:** Code rot and lack of reproducibility if researchers aren't held to minimum code-quality standards. **Priority:** P2 **Estimated Engineering Effort:** Ongoing. **Future Roadmap:** Hosting public weights, full experiment tracking logs, and reproducible run scripts for major academic milestones.